

Question 1: Explain the different file modes available in Python's `open()` function ("`r`", "`w`", "`a`", "`x`", "`r+`", and others), including what happens for each mode if the target file does not already exist. Then explain the difference between `.read()`, `.readline()`, `.readlines()`, and looping directly over a file object, including which approach is most memory-efficient for very large files and why. Support your answer with a short example that opens a text file and reads its content using two different methods.

Answer :- 1. File Modes in Python's `open()` Function

Python's `open(file, mode)` function uses file modes to specify how a file will be accessed and handled.

Mode	Purpose	If File Does Not Exist	If File Exists
"r"	Read (Default). Opens a file for reading only.	Raises <code>FileNotFoundError</code> .	Opens file for reading starting from the beginning (index 0).
"w"	Write. Opens a file for writing only.	Creates a new empty file.	Truncates (overwrites/erases) the existing content completely.

"a"	Append. Opens a file for writing, appending to the end.	Creates a new empty file.	Preserves existing content; places the pointer at the end of the file.
"x"	Exclusive Creation. Creates a new file exclusively for writing.	Creates a new empty file.	Raises FileExistsError to prevent accidental overwriting.
"r+"	Read + Write. Allows reading and modifying an existing file.	Raises FileNotFoundError or.	Opens file for reading/writing without truncating; pointer starts at index 0.
"w+ "	Write + Read. Clears file content and allows read/write access.	Creates a new empty file.	Truncates (erases) the content immediately upon opening.

"a+"	Append + Read. Allows reading and appending new data.	Creates a new empty file.	Preserves existing content; new writes are appended to the end.
"b"	Binary Mode (e.g., "rb", "wb"). Used for non-text files.	Dependent on paired mode (r, w, etc.).	Reads or writes data as raw bytes (e.g., images, PDFs, audio).
"t"	Text Mode (Default) (e.g., "rt", "wt"). Used for text files.	Dependent on paired mode (r, w, etc.).	Automatically decodes raw bytes into standard unicode strings.

2. Differences Between File Reading Methods

- **.read(size=-1)**: Reads the entire contents of the file (or up to **size** bytes/characters if specified) into a single string.
 - *Drawback*: Memory usage grows directly with file size because all content is loaded into RAM at once.
- **.readline(size=-1)**: Reads a single line from the file up to the next newline (**\n**) character and returns it as a string.
- **.readlines()**: Reads all remaining lines from the file and returns them as a list of individual string lines.
 - *Drawback*: Like **.read()**, it loads the entire file into memory simultaneously.
- **Looping directly over the file object (for line in file:)**: Iterates through the file line by line using lazy evaluation (stream buffering).

Most Memory-Efficient Approach for Very Large Files:

Looping directly over the file object (**for line in file:**) is the most memory-efficient approach.

- **Why?** Python uses an internal buffer to stream lines from disk dynamically rather than loading the entire file into RAM. This ensures memory usage remains low and constant (typically $O(1)$ space complexity relative to total file size), preventing **MemoryError** crashes on multi-gigabyte files.

3. Code Example

The code below creates a sample text file and demonstrates reading it using **.read()** and **looping directly over the file object**:

Python

```
# Create a sample text file
```

```
with open("example.txt", "w") as file:
```

```
file.write("Line 1: Python File Handling\n")
```

```
file.write("Line 2: Memory-Efficient Reading\n")
```

```
file.write("Line 3: End of file\n")
```

```
# --- Method 1: Using .read() ---
```

```
print("--- Method 1: Reading entire file using .read() ---")
```

```
with open("example.txt", "r") as file:
```

```
    content = file.read()
```

```
    print(content)
```

```
# --- Method 2: Direct iteration over file object (Memory-Efficient) ---
```

```
print("--- Method 2: Reading line-by-line using direct loop ---")
```

```
with open("example.txt", "r") as file:
```

```
    for line_num, line in enumerate(file, start=1):
```

```
        print(f"Line {line_num}: {line.strip()}")
```

Question 2: Explain the difference between opening a file in “w” mode and “a” mode, and why “w” mode can accidentally destroy existing data if used carelessly. Then explain why every file you open should eventually be closed, what can go wrong if you rely only on a manual `f.close()` call, and how the `with` statement solves this problem — even when an exception is raised inside the block. Write a short program that writes a list of at least three strings to a file using `write()` or `writelines()`, and then appends one more line to the same file without erasing the earlier content.

Answer :- 1. Difference Between "w" Mode and "a" Mode

- **"w" (Write Mode):** Opens a file strictly for writing. If the file exists, Python immediately **truncates (erases)** all existing contents to zero length before any writing occurs. If the file does not exist, a new empty file is created.
- **"a" (Append Mode):** Opens a file for writing without erasing existing content. The file pointer is automatically positioned at the **very end** of the file, ensuring new data is added after existing data. If the file does not exist, a new file is created.

Why "w" Mode Can Destroy Data:

Because "w" mode truncates the file immediately upon execution of `open()`, opening a file in "w" mode by mistake (e.g., intending to read or update log files) will permanently wipe all previously stored data without prompting a warning or asking for confirmation.

2. File Closing, Manual `f.close()`, and the `with` Statement

Why Files Must Be Closed:

When a file is opened, the operating system allocates system resources (a file descriptor) and holds unwritten data in a temporary memory buffer. Closing a file flushes buffered data to the actual disk storage and releases the file descriptor back to the operating system.

Risks of Relying Only on Manual `f.close()`:

1. **Unreleased Resources / Missing Data:** If an error or exception occurs in the code *before* `f.close()` is reached, the close call is skipped entirely. This can cause data loss due to unflushed buffers or leave files locked by the OS.

2. **Code Maintainability:** It is easy for developers to forget to add `f.close()`, especially in complex code with multiple `return` statements or conditional branches.

How the `with` Statement Solves This:

The `with` statement utilizes Python's **Context Manager protocol** (`__enter__` and `__exit__` methods).

- When entering the `with` block, the file is opened (`__enter__`).
- When exiting the `with` block—whether execution completes normally, hits a `return`, or fails due to an unhandled exception—the `__exit__` method is automatically triggered, guaranteeing that `f.close()` is executed immediately and safely.

3. Python Code Example

Python

```
# List of initial strings to write
```

```
initial_data = [  
    "Line 1: First entry in the document.\n",  
    "Line 2: Second entry in the document.\n",  
    "Line 3: Third entry in the document.\n"  
]
```

Step 1: Write initial list of strings using "w" mode and context manager

with open("output.txt", "w") as file:

```
    file.writelines(initial_data)
```

Step 2: Append a new line using "a" mode without erasing earlier content

with open("output.txt", "a") as file:

```
    file.write("Line 4: Appended fourth entry without overwriting.\n")
```

Step 3: Verify and print the final file contents

with open("output.txt", "r") as file:

```
    print("--- Final File Contents ---")
```

```
    print(file.read())
```

Question 3: Explain the difference between a relative path and an absolute path when opening a file. In the specific context of Google Colab, explain where a file created with just a filename, such as `open("sample.txt", "w")`, is actually saved, why that file disappears when the runtime resets, and how mounting Google Drive changes this behaviour. Write a short program that creates a file at a normal (non-Drive) path, writes some content to it, and then reads and prints that content back using a with block.

Answer :- . Relative Path vs. Absolute Path

- **Relative Path:** Specifies the location of a file relative to the **current working directory (CWD)** where the script is currently running.
 - *Example:* `"data/sample.txt"` or `"sample.txt"`.
 - *Pros/Cons:* Shorter and highly portable across different systems, but can fail if the script is executed from a different directory than expected.
- **Absolute Path:** Specifies the complete, full location of a file starting from the system's root directory.
 - *Example (Linux/macOS/Colab):*
`"/content/sample.txt"`
 - *Example (Windows):*
`"C:\\Users\\Username\\Documents\\sample.txt"`
 - *Pros/Cons:* Always points to the exact same file location regardless of the script's execution directory, but hardcodes path structures, making the script less portable across different machines.

2. File Behavior in Google Colab

Where the file is saved:

When you run `open("sample.txt", "w")` in Google Colab without specifying a path, the file is saved in the virtual machine's current working directory, which defaults to `/content/` (the temporary root environment of the active Colab runtime).

Why the file disappears when the runtime resets:

Google Colab provisions a **temporary, ephemeral** hosted virtual machine (**container**) in the cloud for your execution session. When you disconnect, recycle the runtime, or hit the idle timeout limit, that virtual machine instance is completely destroyed and replaced with a fresh environment, wiping all local data saved under `/content/`.

How mounting Google Drive changes this:

By mounting Google Drive (`from google.colab import drive; drive.mount('/content/drive')`), you link persistent cloud storage from your Google Drive account to the Colab virtual machine file system (accessible at `/content/drive/MyDrive/`). Files saved inside this directory persist across session resets, restarts, and runtime disconnects because they reside directly on your cloud drive.

3. Python Code Example

This program creates a text file at a standard non-Drive path (`/content/sample.txt`), writes multiple lines of content to it, and reads it back using a `with` block:

Python

```
# File path definition (normal non-Drive path in local/Colab environment)
```

```
file_path = "sample.txt" # Relative path (defaults to /content/sample.txt in Colab)
```

```
# Step 1: Write content to the file using a context manager
```

```
with open(file_path, "w") as file:
```

```
file.write("Header: Google Colab File Handling Demo\n")  
  
file.write("Line 1: Storing temporary data in local runtime.\n")  
  
file.write("Line 2: This file will persist until the runtime resets.\n")
```

Step 2: Read and print the file content using a context manager

with open(file_path, "r") as file:

```
    content = file.read()  
  
    print("--- File Contents Read Successfully ---")  
  
    print(content)
```

Question 4: Look at the code below and answer the questions:

```
def divide(a, b):  
    try:  
        result = a / b  
    except ZeroDivisionError:  
        print("Cannot divide by zero!")  
    else:  
        print("Division successful:", result)  
    finally:  
        print("Divide operation attempted.")  
divide(10, 2)  
divide(10, 0)
```

Explain the difference between syntax errors, runtime errors (exceptions), and logical errors in Python, giving one short example of each. Then explain exactly what will be printed for each call to `divide()` above, and describe precisely when the `else` block runs compared to when the `finally` block runs.

1. Types of Errors in Python

- **Syntax Errors:** Occur when code violates Python's language rules and structure. These are detected during the **parsing phase** before the code executes, preventing execution entirely.
 - *Example:* `if True` (missing a colon `:` at the end of the line).
- **Runtime Errors (Exceptions):** Occur while the program is actively running when an invalid operation is performed. The syntax is valid, but an issue arises dynamically during execution.
 - *Example:* `10 / 0` (raises `ZeroDivisionError`) or `int("hello")` (raises `ValueError`).
- **Logical Errors:** Occur when the program runs without throwing any errors, but produces incorrect results due to a flaw in algorithm design or math logic.
 - *Example:* `avg = a + b / 2` (missing parentheses `(a + b) / 2`, leading to incorrect mathematical precedence).

2. Exact Output Printed for Each Call

Output for `divide(10, 2)`:

Plaintext

Division successful: 5.0

Divide operation attempted.

- **Explanation:** $10/2$ evaluates to `5.0` without raising an exception. Execution skips the `except` block and enters the `else` block to print `"Division successful: 5.0"`. Finally, the `finally` block executes to print `"Divide operation attempted."`.

Output for `divide(10, 0)`:

Plaintext

Cannot divide by zero!

Divide operation attempted.

- **Explanation:** $10/0$ raises a `ZeroDivisionError`. Execution jumps straight to the `except ZeroDivisionError:` block, printing `"Cannot divide by zero!"`. The `else` block is skipped entirely, and the `finally` block executes to print `"Divide operation attempted."`.

3. When `else` Block Runs vs. When `finally` Block Runs

- **`else` Block:** Runs **only if NO exception was raised** inside the `try` block. If an exception occurs (whether caught or uncaught), the `else` block is completely skipped.
- **`finally` Block:** Runs **always and unconditionally**, regardless of whether an exception occurred, whether it was handled, or if a statement like `return`, `break`, or `raise` was executed inside the earlier blocks. It is primarily used for cleanup tasks like releasing resources or closing files.

Question 5: Explain why using a bare `except:` is considered bad practice in real projects, and why you should always try to catch the most specific exception possible. Then create a custom exception class called `InsufficientFundsError` that inherits from `Exception`, and write a function `withdraw(balance, amount)` that raises this custom exception (with a meaningful message) whenever `amount` is greater than `balance`. Demonstrate calling this function inside a `try/except` block that specifically catches `InsufficientFundsError`, and show what gets printed both when a withdrawal succeeds and when it fails.

Answer :- Why a Bare `except:` is Bad Practice:

1. **Catches System Signals:** A bare `except:` catches every exception derived from `BaseException`, including critical system interrupts like `KeyboardInterrupt` (triggered when a user presses Ctrl+C to stop execution) and `SystemExit`. This makes it difficult to stop or kill a running program.
2. **Masks Unexpected Bugs:** It catches unrelated errors like `NameError` (typos in variable names), `TypeError`, or `AttributeError`. Instead of crashing immediately so developers can identify and fix bugs, a bare `except:` suppresses these errors silently.
3. **Complicates Debugging:** By masking the underlying issue and suppressing traceback details, identifying where and why a failure occurred in a production system becomes significantly harder.

Why You Should Catch Specific Exceptions:

Catching specific exceptions (e.g., `FileNotFoundError`, `ValueError`, or custom domain errors) guarantees that your application only handles known, recoverable error scenarios while letting critical bugs or termination signals propagate naturally for proper logging and debugging.

2. Custom Exception and `withdraw()` Function Implementation

Python

Step 1: Create a custom exception inheriting from Exception

```
class InsufficientFundsError(Exception):
```

```
    """Raised when the withdrawal amount exceeds the current account
    balance."""
```

```
    pass
```

Step 2: Define the withdraw function

```
def withdraw(balance, amount):
```

```
    if amount > balance:
```

```
        raise InsufficientFundsError(
```

```
            f'Insufficient funds! Requested: ${amount:.2f}, Available:
            ${balance:.2f}'
```

```
        )
```

```
balance -= amount
```

```
return balance
```

```
# --- Step 3: Demonstration ---
```

```
# Scenario A: Successful Withdrawal
```

```
print("--- Scenario 1: Successful Withdrawal ---")
```

```
current_balance = 500.00
```

```
withdrawal_amount = 200.00
```

```
try:
```

```
    current_balance = withdraw(current_balance, withdrawal_amount)
```

```
    print(f"Withdrawal successful! New balance:  
    ${current_balance:.2f}")
```

```
except InsufficientFundsError as e:
```

```
    print(f"Transaction Failed: {e}")
```

```
print()
```

```
# Scenario B: Failed Withdrawal (Insufficient Funds)

print("--- Scenario 2: Failed Withdrawal ---")

current_balance = 300.00

withdrawal_amount = 500.00

try:

    current_balance = withdraw(current_balance, withdrawal_amount)

    print(f"Withdrawal successful! New balance:
    ${current_balance:.2f}")

except InsufficientFundsError as e:

    print(f"Transaction Failed: {e}")
```

3. Expected Printed Output

Plaintext

--- Scenario 1: Successful Withdrawal ---

Withdrawal successful! New balance: \$300.00

--- Scenario 2: Failed Withdrawal ---

Transaction Failed: Insufficient funds! Requested: \$500.00, Av

Question 6: Explain at least three advantages of using Python's logging module over plain `print()` statements for tracking what a program is doing. List the five standard log levels (DEBUG, INFO, WARNING, ERROR, CRITICAL) in order of increasing severity, and give one realistic example of when you would use each. Then write a short program that uses `logging.basicConfig()` to write log messages of at least three different levels to a file called `app.log`, with each log line including a timestamp and the severity level.

Answer :- 1. Advantages of **logging** Over **print()**

1. **Configurable Log Levels & Filtering:** The **logging** module allows you to filter messages dynamically based on severity (e.g., show only **WARNING** and higher in production) without deleting or commenting out debug messages in code.
2. **Flexible Destinations:** Outputs can be easily redirected to files, standard output, remote servers, or external monitoring systems (like Sentry or Syslog) by changing configuration, whereas **print()** defaults strictly to standard output (**stdout**).
3. **Structured Metadata & Formatting:** Automatically captures crucial context such as timestamps, severity levels, file names, line numbers, and thread/process IDs without manual string formatting.

2. Standard Log Levels (In Order of Increasing Severity)

1. **DEBUG (Severity: 10):** Detailed diagnostic information used for troubleshooting during development.

- *Example:* Logging raw database query strings or variable values inside a loop.
- 2. **INFO (Severity: 20):** Confirmation that tasks are proceeding as expected during routine operation.
 - *Example:* Logging when a user successfully logs in or a batch job completes.
- 3. **WARNING (Severity: 30):** An unexpected event occurred or a potential issue might arise, but the application is still functioning normally.
 - *Example:* Disk space running low or using a deprecated API function.
- 4. **ERROR (Severity: 40):** A serious issue occurred that prevented a specific function or operation from executing properly.
 - *Example:* Database connection failed or a critical configuration file was not found.
- 5. **CRITICAL (Severity: 50):** A fatal error occurred indicating that the entire program or application may crash or terminate unexpectedly.
 - *Example:* Out of memory failure or payment gateway complete outage.

3. Python Code Example

Python

```
import logging
```

```
# Configure logging to output to app.log with timestamp and severity level
```

```
logging.basicConfig(
```

```
filename="app.log",

filemode="w",

level=logging.DEBUG,

format="%(asctime)s - %(levelname)s - %(message)s",

datefmt="%Y-%m-%d %H:%M:%S"

)

# Logging messages at three different severity levels

logging.info("Application starting up and initializing services.")

logging.warning("High CPU utilization detected (>85%).")

logging.error("Failed to fetch data from external API endpoint.")

print("Log messages written to app.log successfully.")
```

Question 7: Look at the code below and answer the questions:

```
import logging
logging.basicConfig(filename="errors.log",
                    level=logging.DEBUG)
def load_config(path):
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError:
```

```
        logging.exception("Failed to load config file")
    return None
```

```
load_config("does_not_exist.cfg")
```

Explain what gets written to

errors.log when load_config() is called on a path that doesn't

exist. Then explain the difference between logging.error() and

logging.exception() — specifically, what extra information

logging.exception() records that logging.error() does not, and why

it should only ever be called from inside an except block.

Answer :- When `load_config("does_not_exist.cfg")` is executed, opening the non-existent file triggers a `FileNotFoundError`. The `except FileNotFoundError:` block catches this error and calls `logging.exception("Failed to load config file")`.

`errors.log` receives a log entry at the **ERROR** severity level containing:

1. The log level and custom message: **ERROR:root:Failed to load config file**.
2. The complete **Exception Traceback**, detailing the line of code that threw the error (`open(path)`), the exception type (`FileNotFoundError`), and the system error details (`[Errno 2] No such file or directory: 'does_not_exist.cfg'`).

Exact Contents of `errors.log`:

Plaintext

ERROR:root:Failed to load config file

Traceback (most recent call last):

File "script.py", line 5, in load_config

with open(path) as f:

FileNotFoundError: [Errno 2] No such file or directory:
'does_not_exist.cfg'

2. Difference Between `logging.error()` and `logging.exception()`

Feature	<code>logging.error()</code>	<code>logging.exception()</code>

Log Severity Level	ERROR	ERROR
Traceback Inclusion	No (by default). Only logs the provided string message unless exc_info=True is passed explicitly.	Yes (automatically). Automatically attaches the full exception traceback.
Where to Call	Can be called anywhere in your codebase.	Should only be called from within an except block.

The Extra Information `logging.exception()` Records:

`logging.exception()` automatically extracts and appends the **full stack traceback** (`sys.exc_info()`) to the log entry. This includes:

- The exact chain of function calls leading to the error.
- The specific line number and source file where the exception occurred.
- The formal exception class name and error message string.

Why `logging.exception()` Must Only Be Called Inside an **except** Block:

`logging.exception()` relies on Python's active exception context (`sys.exc_info()`) to extract traceback details. If called outside an active **except** block where no exception is currently being handled, `sys.exc_info()` returns `(None, None, None)`. This results in printing `NoneType: None` in place of the traceback, creating misleading or invalid log entries.

Question 8: Describe at least four different ways to create a NumPy array (for example: from a Python list, and using `np.zeros()`, `np.ones()`, `np.full()`, `np.eye()`, `np.arange()`, or `np.linspace()`), including one realistic data-science or machine-learning situation where each would be useful. Then explain what each of the following array attributes tells you: `shape`, `ndim`, `size`, `dtype`, and `itemsize`. Support your answer with a short example that creates a 2D array and prints all five of these attributes for it.

Answer :- . **Four Ways to Create a NumPy Array & Practical Use Cases**

1. From a Python List (`np.array()`)

Converts a standard Python list or nested list into a single- or multi-dimensional NumPy array.

- **Realistic Use Case:** Converting raw tabular data, user input lists, or JSON/CSV records parsed in Python into a numerical structure for data preprocessing.

2. Using `np.zeros()`

Creates an array of a specified shape filled entirely with zeros (`0.0`).

- **Realistic Use Case:** Pre-allocating memory for large output matrices, gradient accumulators, or initializing weight vectors before iteratively updating them inside a loop.

3. Using `np.arange()`

Generates an array with evenly spaced values within a specified interval [`start`, `stop`) given a step size.

- **Realistic Use Case:** Generating time steps, epoch indices, or positional encodings for time-series models and data visualizations.

4. Using `np.linspace()`

Generates `N` evenly spaced numbers over a specified closed range [`start`, `stop`].

- **Realistic Use Case:** Constructing a continuous feature grid to evaluate loss surfaces, plot decision boundaries, or perform parameter grid searches.

2. NumPy Array Attributes Explained

- **shape:** Returns a tuple of integers indicating the dimensions (size along each axis) of the array (e.g., (`rows`, `columns`)).

- **ndim**: Returns an integer representing the total number of dimensions/axes of the array (e.g., **1** for a 1D vector, **2** for a 2D matrix).
- **size**: Returns an integer representing the total number of elements in the entire array (equal to the product of all values in **shape**).
- **dtype**: Specifies the data type of the array elements (e.g., **int64**, **float64**, **bool**).
- **itemsize**: Returns the size in bytes of a single array element (e.g., **8** bytes for a standard 64-bit integer or float).

3. Code Example

Python

```
import numpy as np
```

```
# Create a 2D NumPy array (2 rows, 3 columns)
```

```
matrix = np.array([
```

```
    [10, 20, 30],
```

```
    [40, 50, 60]
```

```
], dtype=np.int64)
```

```
# Print all 5 required attributes
```

```
print("Array Attributes:")
```

```
print(f"shape: {matrix.shape}") # (2, 3)

print(f"ndim: {matrix.ndim}") # 2

print(f"size: {matrix.size}") # 6

print(f"dtype: {matrix.dtype}") # int64

print(f"itemsize: {matrix.itemsize}") # 8 bytes
```

Output:

Plaintext

Array Attributes:

shape: (2, 3)

ndim: 2

size: 6

dtype: int64

itemsize: 8

Question 9: Explain the difference between basic slicing, boolean indexing, and fancy (integer-array) indexing in NumPy — specifically, which of these return a view of the original array and which return a copy. Using a short code example, demonstrate a case where modifying a sliced array also changes the original array, and explain how `.copy()` can be used to avoid this. Why does this view-versus-copy distinction matter when working with large datasets?

Answer :- 1. **Basic Slicing vs. Boolean Indexing vs. Fancy Indexing**

- **Basic Slicing:** Uses slice notation `[start:stop:step]` to extract continuous or regularly strided sub-regions of an array.
 - **Memory Return:** Returns a **VIEW** of the original array (shares the exact same memory buffer in RAM).
- **Boolean Indexing:** Uses boolean arrays or conditions (e.g., `arr[arr > 5]`) to select elements where the conditional statement evaluates to `True`.
 - **Memory Return:** Returns a **COPY** of the data (allocates a new memory buffer).
- **Fancy (Integer-Array) Indexing:** Uses lists or arrays of explicit integer indices (e.g., `arr[[0, 2, 4]]`) to extract non-contiguous elements or specific rows/columns.
 - **Memory Return:** Returns a **COPY** of the data (allocates a new memory buffer).

2. **Code Example: View Modification vs. Using `.copy()`**

Python

```
import numpy as np
```

```
# Create original array
```

```
original = np.array([10, 20, 30, 40, 50])
```

```
# --- Demonstration 1: Slicing returns a VIEW ---
```

```
slice_view = original[1:4] # Slices elements [20, 30, 40]
```

```
slice_view[0] = 999      # Modify the view
```

```
print("Original array after modifying view:", original)
```

```
# Output: [ 10 999  30  40  50] -> The original array was modified!
```

```
# --- Demonstration 2: Using .copy() to prevent unintended  
modification ---
```

```
original_reset = np.array([10, 20, 30, 40, 50])
```

```
slice_copy = original_reset[1:4].copy() # Explicit copy
```

```
slice_copy[0] = 999      # Modify the copy
```

```
print("Original array after modifying copy:", original_reset)
```

Output: [10 20 30 40 50] -> Original remains untouched!

3. Why the View vs. Copy Distinction Matters with Large Datasets

1. **Memory Overhead:** A **view** creates a lightweight reference object in $O(1)$ time without duplicating memory. For multi-gigabyte datasets, slicing via views avoids multiplying your memory footprint.
2. **Data Integrity & Side Effects:** Modifying a slice returned as a **view** accidentally mutates the source array. In machine learning pipelines, modifying a training feature slice without realizing it is a view can corrupt your original dataset.
3. **Computation Speed:** Unintentionally triggering **copies** (via fancy indexing or explicit `.copy()` calls) forces CPU-intensive data duplication across RAM, creating performance bottlenecks on massive matrices.

Question 10: Explain NumPy's broadcasting rules in your own words, and give a short example of adding a 1D array to a 2D array where broadcasting is used. Then explain the difference between `np.sum(arr, axis=0)` and `np.sum(arr, axis=1)` for a 2D array, with a short example of each. Finally, explain the difference between `reshape()`, `flatten()`, `np.hstack()`, and `np.vstack()`, and demonstrate creating a 1D array of 6 elements, reshaping it into a 2×3 array, and stacking two such arrays vertically using `np.vstack()`.

Answer :- 1. NumPy's Broadcasting Rules

Broadcasting is NumPy's mechanism for performing arithmetic operations on arrays of different shapes without making unnecessary copies of data in memory.

To determine if two arrays are compatible for broadcasting, NumPy compares their shapes **element-wise, starting from the trailing (rightmost) dimensions** and moving left:

1. **Rule 1 (Dimension Alignment):** If the arrays have a different number of dimensions, the shape of the smaller array is padded with **1**s on its left side.
2. **Rule 2 (Compatibility Check):** Two dimensions are compatible if:
 - They are equal, **OR**
 - One of the dimensions is **1**.
3. **Rule 3 (Expansion):** If a dimension has a size of **1**, it is logically stretched or repeated along that axis to match the larger dimension. If the dimensions do not match and neither is **1**, a **ValueError** is raised.

Example: Adding a 1D Array to a 2D Array

Python

```
import numpy as np
```

```
# 2D array of shape (2, 3)
```

```
matrix = np.array([
```

```
    [10, 20, 30],
```

```
[40, 50, 60]
```

```
])
```

```
# 1D array of shape (3,) -> NumPy treats its shape as (1, 3) for  
broadcasting
```

```
vector = np.array([1, 2, 3])
```

```
# Adding matrix (2, 3) and vector (1, 3)
```

```
result = matrix + vector
```

```
print("Broadcasting Result:\n", result)
```

```
# Output:
```

```
# [[11 22 33]
```

```
# [41 52 63]]
```

2. Difference Between `np.sum(arr, axis=0)` and `np.sum(arr, axis=1)`

In a 2D array:

- **axis=0** refers to the **vertical axis (rows)**. Summing along **axis=0** collapses the rows and calculates the **column-wise totals**, returning a 1D array with a length equal to the number of columns.
- **axis=1** refers to the **horizontal axis (columns)**. Summing along **axis=1** collapses the columns and calculates the **row-wise totals**, returning a 1D array with a length equal to the number of rows.

Example:

Python

```
arr = np.array([  
  
    [1, 2, 3],  
  
    [4, 5, 6]  
  
])
```

```
# Sum along axis=0 (Column-wise sum: [1+4, 2+5, 3+6])
```

```
col_sum = np.sum(arr, axis=0)
```

```
print("Column-wise sum (axis=0):", col_sum) # [5 7 9]
```

```
# Sum along axis=1 (Row-wise sum: [1+2+3, 4+5+6])
```

```
row_sum = np.sum(arr, axis=1)
```

```
print("Row-wise sum (axis=1):", row_sum) # [6 15]
```

3. Differences Between Array Manipulation Functions

- **reshape()**: Changes the dimensions (shape) of an array without altering its data or total number of elements.
- **flatten()**: Collapses a multi-dimensional array into a 1D vector. It returns a **copy** of the array in new memory.
- **np.hstack()**: Stacks arrays **horizontally** (side-by-side along **axis=1**), increasing the number of columns.
- **np.vstack()**: Stacks arrays **vertically** (row upon row along **axis=0**), increasing the number of rows.

4. Code Example: Reshaping and Stacking

Python

```
import numpy as np
```

```
# Step 1: Create a 1D array of 6 elements
```

```
arr_1d = np.array([1, 2, 3, 4, 5, 6])
```

```
print("Original 1D array:", arr_1d)
```

```
# Step 2: Reshape into a 2x3 array
```

```
arr_2x3 = arr_1d.reshape(2, 3)
```

```
print("\nReshaped 2x3 array:\n", arr_2x3)
```

```
# Step 3: Stack two such arrays vertically using np.vstack()

stacked_arr = np.vstack((arr_2x3, arr_2x3))

print("\nVertically stacked array (4x3):\n", stacked_arr)
```

Output:

Plaintext

Original 1D array: [1 2 3 4 5 6]

Reshaped 2x3 array:

[[1 2 3]

[4 5 6]]

Vertically stacked array (4x3):

[[1 2 3]

[4 5 6]

[1 2 3]

[4 5 6]]

